



Module 5 - Prompt Library

Practical prompts for Codex App review, governance, planning, context engineering, and personal AI playbooks

AI-Assisted Coding

How to use this library

- Use this library while completing Module 5 in Codex App. The default workflow is read first, inspect, grade, and only then decide what belongs in docs/.
- Replace placeholders such as [PASTE AI FINDINGS HERE] or [PASTE YOUR NOTES HERE]. Do not leave placeholders in a final artifact.
- Start fresh threads for separate tasks, especially when comparing generic vs repo-grounded plans or context strategy A/B/C.
- Ask Codex to cite actual files when it makes claims about the Task Tracker repo. Generic advice is not enough for Module 5.
- Keep Module 5 outputs in docs/. Reject unexpected edits to app/ unless you intentionally choose the optional one-line fix from the security review.
- Treat every answer as a draft to grade. Strong prompts ask for evidence, constraints, output format, and a verification or critique step.

Key mindset: AI proposes; you grade. In Module 5, the prompt is not just a request for output; it is a tool for creating evidence you can inspect, compare, and turn into rules.

Setup prompts - Part 5.1

These prompts establish the Module 5 posture: Codex App should read the repo, respect AGENTS.md, avoid hidden edits, and treat docs/ as the working area for this module.

5.1A - Start a Codex App thread with Module 5 guardrails

Prompt category: Context / task boundary

When to use it: Use at the beginning of a fresh Codex App thread before any Module 5 activity.

Weak version:

Remember this is Module 5. Help me with the repo.

Improved version:

I am working in Module 5 of AI-Assisted Coding with the Task Tracker repository in Codex App.

Module goal:

I am grading and governing AI-assisted coding work. I am not building new app features in this module.

Default constraints:

- Prefer read-only analysis first.
- Edit files in docs/ only unless I explicitly approve a different path.
- Do not modify app/ during Module 5 unless I ask for one specific minimal fix.
- Use one bounded task per thread.
- When you make claims about the repo, cite actual files you inspected.
- If you are uncertain or a file is not visible, say so instead of guessing.

Before answering, briefly state:

1. The task you think I am asking for.
2. The files you plan to inspect, if any.
3. Whether you need permission to edit anything.

Why the improved version is better: It gives Codex the module purpose, the review posture, the docs-first boundary, and the requirement for file evidence. This prevents the agent from treating Module 5 like another feature-building module.

Optional follow-up prompt:

Before you continue, repeat the Module 5 guardrails in your own words and tell me what you will not change unless I explicitly approve it.

Common mistake this prompt avoids: Starting a long agent thread with vague memory and then being surprised when it edits the wrong part of the repo.

5.1B - Draft or review AGENTS.md

Prompt category: Repo-level instructions / Codex App setup

When to use it: Use after opening the Task Tracker repo in Codex App and before running the rest of Module 5.

Weak version:

Create AGENTS.md for this repo.

Improved version:

You are helping me set up Codex App for the Module 5 Task Tracker repo.

Task:

Draft an AGENTS.md file for this repository. Read the repo first, especially README.md, app/main.py, app/models.py, app/storage.py or equivalent persistence files, tests/, frontend/index.html if present, and requirements.txt or pyproject.toml if present.

AGENTS.md must include:

1. Project summary.
2. Tech stack and run/test commands that are actually supported by the repo.
3. Business rules that are visible in the code, such as task statuses, priorities, and validation rules.
4. Module 5 guardrails: docs-first, read-only by default, one task per thread, and no app/ changes unless explicitly approved.
5. Security and governance reminders: do not paste or expose secrets, do not run destructive commands, cite files, and do not invent

findings.

Constraints:

- Do not edit any file except AGENTS.md.
- Show me the proposed AGENTS.md content and list the files you read before applying the change.
- If a command or business rule is not visible, mark it as "not confirmed" instead of inventing it.

Output format:

1. Files inspected
2. Proposed AGENTS.md
3. Assumptions or items to verify
4. Diff preview if you plan to create or update the file

Why the improved version is better: The old project-instructions idea is useful, but Module 5 now uses AGENTS.md.

This prompt makes the file repo-grounded instead of generic and keeps Codex inside Module 5 boundaries.

Optional follow-up prompt:

Review the proposed AGENTS.md against the Module 5 guardrails. Flag any instruction that is too broad, too risky, or not supported by the repo.

Common mistake this prompt avoids: Creating a polished AGENTS.md that sounds helpful but does not reflect the actual repo or the Module 5 docs-first workflow.

5.1C - Project evidence smoke test

Prompt category: Repo grounding / file evidence

When to use it: Use immediately after selecting the Task Tracker repo in Codex App to verify the agent is reading real files.

Weak version:

Summarize this project.

Improved version:

Summarize the Task Tracker repo in 5 bullets.

For each bullet:

- Name the exact file or files you inspected.
- Quote or paraphrase the evidence briefly.
- State your confidence level: High, Medium, or Low.

Constraints:

- Do not edit files.
- Do not run tests or start the app.
- Do not rely on generic FastAPI assumptions.
- If you cannot find evidence for a claim, do not include the claim.

Output format:

Return a table with columns: Claim, Evidence file, Evidence summary, Confidence, Assumption to verify.

Why the improved version is better: It turns a generic repo summary into an evidence check. Students can quickly see whether Codex is grounded in their actual files.

Optional follow-up prompt:

Which of your claims would change if this repo used a different storage approach? Point me to the file that supports your answer.

Common mistake this prompt avoids: Accepting a fluent project summary that could describe any Task Tracker app.

5.1D - Recent files smoke test

Prompt category: Repo grounding / filesystem awareness

When to use it: Use after the project evidence smoke test to confirm Codex can inspect local repo state without editing files.

Weak version:

What changed recently?

Improved version:

Using filesystem metadata only, identify the three most recently modified files in this repo.

For each file, provide:

- File path
- Last modified time if visible
- One sentence describing what you see in the file

Constraints:

- Do not use git.
- Do not edit anything.
- Do not infer from filenames alone. Open or inspect the file before describing it.
- If filesystem metadata is unavailable, say that clearly and suggest a safe alternative read-only check.

Output format:

Return a table with columns: File, Modified time, What the file contains, Evidence confidence.

Why the improved version is better: The lecture uses smoke tests to confirm Codex is looking at the real repo. This prompt checks repo access without encouraging edits or commands that change state.

Optional follow-up prompt:

Pick one of those files and tell me one specific detail that proves you opened it, not just read the filename.

Common mistake this prompt avoids: Continuing into security review or planning before verifying that the selected folder is actually the repo root.

Security review prompts - Part 5.2

These prompts support the first major grading exercise. The goal is docs/security-review.md, not a code patch. The strongest student work appears in the grading, manual scan, You-only findings, and top-3 backlog.

5.2A - Run a read-only security audit

Prompt category: Security review / file-evidence audit

When to use it: Use in a fresh Codex App thread after AGENTS.md and the smoke tests are complete.

Weak version:

Review this codebase for security issues.

Improved version:

You are a security-minded reviewer doing a read-only audit of the Task Tracker repo in Codex App.

Task:

Inspect relevant files before answering, including:

- Backend app files such as app/main.py, route files, models, validation, and storage.
- tests/ to understand expected behavior.
- requirements.txt or pyproject.toml if present.
- Dockerfile, compose files, or .github workflows if present.
- frontend/index.html or frontend files if present.
- AGENTS.md for project instructions and Module 5 guardrails.

Focus areas:

1. Input validation, unbounded strings, enum handling, and type assumptions.
2. Authorization gaps or the intentional absence of auth as a course-scope decision.
3. Data exposure, stack traces, verbose errors, secrets, or unsafe config.
4. Error handling and broad exception behavior.
5. Dependency, CI, Docker, and deployment risks.
6. CORS or frontend API assumptions.

Constraints:

- Read-only. Do not edit files.
- Do not run destructive commands.
- Do not invent findings to fill space.
- If a category is clean, not applicable, or not visible, say so.
- Cite actual file paths and line numbers if available.

Output format:

Return a markdown table with columns:

ID | Severity | File / location | Finding | Evidence | Suggested next step | Confidence

After the table, add:

- Files inspected
- Categories where you found no issue
- Any assumptions or limits of the audit

Why the improved version is better: It requires Codex to read the repo first, separates evidence from opinion, and prevents generic OWASP-style output from looking more useful than it is.

Optional follow-up prompt:

Before I copy this into docs/security-review.md, identify any finding that may be a course-scope decision rather than a production vulnerability.

Common mistake this prompt avoids: Treating a fluent AI security report as correct without checking whether it cites real files and real behavior.

5.2B - Grade AI security findings

Prompt category: Security review / human grading

When to use it: Use after Codex produces the security audit and before writing the final security-review.md.

Weak version:

Are these findings right?

Improved version:

I am grading AI security findings for Module 5.

Definitions:

- Valid: a real issue in this repo, or a course-scope limitation that would matter outside the learning context.
- False Positive: wrong because of the actual code, severity, behavior, or Task Tracker scope.
- Noise: technically true but too generic, trivial, or unsupported to become an action item.

Task:

For each finding below, help me classify it as Valid, False Positive, or Noise. If you need code or file evidence to grade a finding, ask for it instead of guessing.

Constraints:

- Do not invent new findings.
- Do not upgrade severity without evidence.
- Treat "no authentication" carefully: it may be an intentional course-scope decision, but it may still be a production risk.
- Keep the final judgment mine; provide evidence and reasoning.

Output format:

Return a table with columns:

Finding ID | Proposed grade | Reason | Evidence needed or evidence used | Student decision to confirm

AI findings:

[PASTE AI SECURITY FINDINGS HERE]

Why the improved version is better: The improved prompt matches the lecture principle: the AI output is not the deliverable; the student grading is the deliverable.

Optional follow-up prompt:

For every finding marked Valid, suggest the smallest next action: backlog, documentation note, test, or optional one-line fix.

Common mistake this prompt avoids: Marking every AI finding as Valid because the report sounds professional.

5.2C - Reconcile manual scan with AI findings

Prompt category: Security review / reconciliation

When to use it: Use after completing your own manual scan of app/ and related files.

Weak version:

Find what the AI missed.

Improved version:

I completed a Module 5 security review. Below are the AI findings and my own manual scan findings.

Task:

1. Group the findings into Agreement, AI-only, and You-only.
2. Write a two-line observation about the shape of AI coverage.
3. Create a top-3 security backlog from the Valid findings.

Definitions:

- Agreement: both AI and I noticed the issue or a closely related issue.
- AI-only: AI noticed it and I did not notice it during my manual scan.
- You-only: I noticed it and AI missed it. This is the most important learning column.

Constraints:

- Do not invent new findings.
- If a finding lacks file evidence, mark it "needs evidence" instead of treating it as confirmed.
- Suggested owners should be realistic, such as backend, frontend, DevOps, or course/project owner.

Output format:

1. Reconciliation table with columns: Agreement, AI-only, You-only
2. Two-line observation
3. Top-3 backlog table with columns: Rank, Finding, Why it matters, Suggested owner, Next action

AI findings:

[PASTE AI FINDINGS WITH GRADES HERE]

My manual scan findings:

[PASTE YOUR MANUAL FINDINGS HERE]

Why the improved version is better: It directly supports the lecture artifact: docs/security-review.md with Agreement / AI-only / You-only and a top-3 backlog. It also protects the student-owned manual scan.

Optional follow-up prompt:

Make the You-only column more specific by asking what business logic, scope decision, or threat-model context explains each item.

Common mistake this prompt avoids: Stopping at the AI audit and skipping the manual scan, which removes the main evidence of student judgment.

5.2X - Optional minimal one-line fix

Prompt category: Optional extension / security fix preview

When to use it: Use only if one Valid finding is clearly a one-line fix and you intentionally decide to review a patch.

Weak version:

Fix this security issue.

Improved version:

I want to consider one optional one-line fix from docs/security-review.md.

Finding:

[PASTE THE ONE FINDING HERE]

Task:

Propose the minimal diff that addresses only this finding. Do not apply the diff yet.

Constraints:

- One finding only.
- No refactoring.
- No unrelated validation changes.
- No new dependencies.
- Do not edit tests unless I explicitly ask.
- Explain why this is small enough for Module 5; if it is not small enough, say it belongs in the backlog.

Output format:

1. Decision: one-line fix or backlog
2. Minimal unified diff
3. Why this change is limited
4. Verification step I should run or document

Why the improved version is better: The old minimal-fix prompt was useful, but the recording frames Module 5 as documentation-heavy. This version keeps the option but makes it conservative.

Optional follow-up prompt:

Before I apply this diff, list any possible side effects or tests I should check.

Common mistake this prompt avoids: Turning a security review into a broad refactor and losing the Module 5 focus on grading and documentation.

Governance prompts - Part 5.3

These prompts help students reconstruct what they shared with AI, what they received, what they understand, and what concrete rules they want to follow next time.

5.3A - Classify what you shared with AI

Prompt category: Governance / risk classification

When to use it: Use after filling the "What I Shared" table in docs/governance-worksheet.md.

Weak version:

Classify my AI prompts by risk.

Improved version:

I am doing a Module 5 governance retrospective on what I shared with AI coding tools during this course.

Risk rubric:

- Low: public code, course toy project code, no sensitive data, no proprietary logic.
- Medium: private but non-sensitive code, internal implementation details, or non-public repo context with no secrets and no PII.
- High: credentials, tokens, secrets, production config, real customer/user data, regulated data, or code I am not authorized to share.

Task:

For each row in my "What I Shared" table, classify the risk as Low, Medium, or High. Add a one-sentence reason and a safer future

version of what I could paste instead.

Constraints:

- If a row is ambiguous, say what information is missing instead of guessing.
- Do not minimize risk because the project is small.
- Do not invent rows I did not provide.

Output format:

Return a table with columns:

Item shared | Risk | Reason | Safer future version | Ambiguity to resolve

What I Shared:

[PASTE YOUR TABLE HERE]

Why the improved version is better: It turns governance into a practical classification exercise and adds the "safer future version" students can actually reuse.

Optional follow-up prompt:

Now sort the High and Medium items by which habit I should change first.

Common mistake this prompt avoids: Calling everything Low because the Task Tracker feels like a learning project, even when real names, secrets, or private code were pasted.

5.3B - Trace one generated block line by line

Prompt category: Governance / ownership of generated code

When to use it: Use for one AI-generated code block from Modules 1-4 that you cannot fully explain.

Weak version:

Explain this code.

Improved version:

Walk me through the following AI-generated code block line by line.

For each line or small group of lines, explain:

1. What it does.
2. Why it is written this way.
3. What would break if it were removed or changed.
4. Any assumption, risk, or library behavior I should verify.

Constraints:

- Use beginner-friendly but precise language.
- If the explanation would require seeing surrounding files, ask for them.
- If you are unsure, say so. Do not use vague phrases such as "this is standard" without explaining why.
- Do not rewrite the code unless I ask.

Output format:

Return a table with columns:

Line(s) | What it does | Why it is there | What could break | Do I own this yet?

Code block:

```
```  
[PASTE ONE GENERATED CODE BLOCK HERE]
```
```

Why the improved version is better: The lecture emphasizes that code you cannot explain is code you do not own. This prompt turns understanding into a concrete review table.

Optional follow-up prompt:

Mark the three lines I should understand before committing this code, and ask me one check question for each.

Common mistake this prompt avoids: Assuming a generated block is safe because it ran once or because the explanation sounded confident.

5.3C - Turn notes into concrete AI usage rules

Prompt category: Governance / personal rules

When to use it: Use after completing the governance worksheet and tracing at least one generated code block.

Weak version:

Write my AI policy.

Improved version:

Use only my governance worksheet notes below. Do not invent events, examples, or rules.

Task:

Help me turn my notes into three personal AI usage rules:

1. What I will never paste.
2. What I will always verify before accepting.
3. How I will record AI contributions.

Quality bar:

Each rule must be concrete enough that a teammate could tell whether a future action violates it. If the rule sounds like "be careful," it is too vague.

Constraints:

- Do not write generic company policy.
- Do not add examples that are not in my notes.
- If my notes do not support a rule, write "Missing - add course evidence".
- Keep the language student-friendly and specific.

Output format:

Return a table with columns:

Rule category | Draft rule | Evidence from my notes | What is still vague? | Revised rule

My notes:

[PASTE GOVERNANCE WORKSHEET NOTES HERE]

Why the improved version is better: It preserves student ownership while still using AI to organize and sharpen the rules. The prompt prevents a polished but fictional governance reflection.

Optional follow-up prompt:

For each revised rule, test it against one future scenario and tell me whether the rule gives a clear yes/no decision.

Common mistake this prompt avoids: Letting AI write generic rules that sound responsible but do not come from the student's actual course experience.

Feature planning prompts - Part 5.4

These prompts support the comments-feature planning exercise. The core comparison is generic planning versus repo-grounded planning. Students plan and critique; they do not implement the feature.

5.4A - Create a generic comments feature plan

Prompt category: Feature planning / ungrounded baseline

When to use it: Use in a fresh thread without repo access or pasted code. The goal is to create a baseline to compare against the repo-grounded plan.

Weak version:

Plan comments for my task tracker.

Improved version:

Without reading repo files, produce a generic plan for adding comments to tasks in a Task Tracker app.

A comment has:

- id: string UUID
- task_id: string foreign key or task reference
- author: required string, 1-100 characters
- body: required string, 1-2000 characters
- created_at: server-generated UTC datetime

Task:

Create a plan with these sections:

1. Data Model
2. API Routes
3. Tests
4. Frontend Changes
5. Migration or data-shape notes
6. Open Questions

Constraints:

- Do not write implementation code.
- Clearly label assumptions as assumptions.
- Do not claim knowledge of my actual file names, storage approach, frontend structure, or test style.
- Keep this generic; I will compare it with a repo-grounded Codex App plan.

Output format:

Return a markdown plan followed by a section called "Assumptions this plan makes."

Why the improved version is better: The lecture compares generic and repo-grounded planning. This prompt intentionally creates the generic baseline while forcing the model to label its assumptions.

Optional follow-up prompt:

Which parts of this plan are most likely to be wrong once you read the actual repo?

Common mistake this prompt avoids: Mistaking a coherent generic plan for a plan that fits the actual codebase.

5.4B - Create a repo-grounded comments feature plan

Prompt category: Feature planning / agent-grounded design

When to use it: Use in a fresh Codex App thread after creating the generic plan. This is the main Part 5.4 prompt.

Weak version:

Make the comments plan fit my repo.

Improved version:

Plan - do not implement - a "comments on tasks" feature for this Task Tracker repo.

A comment has:

- id: string UUID
- task_id: string foreign key or task reference
- author: required string, 1-100 characters
- body: required string, 1-2000 characters
- created_at: server-generated UTC datetime

Before planning, read the actual repository files, especially:

- AGENTS.md
- app/models.py or equivalent model files
- app/main.py and any route files
- app/storage.py or wherever tasks are persisted
- tests/ to understand test style
- frontend/index.html or equivalent frontend files
- README.md if it documents run behavior or conventions

Task:

Produce a design document with these sections:

1. Data Model - where the comment models belong and how they fit existing patterns.
2. API Routes - HTTP method, path, request body, response body, and error cases.
3. Tests - concrete test names grouped by happy path, validation, and edge cases.
4. Frontend Changes - which files would change and what the user would see.
5. Migration Notes - anything about existing task data or storage shape that must change.
6. Open Questions - at least three real decisions the team should make.

Constraints:

- Read-only. Do not edit app/ or create implementation files.
- Do not produce implementation code blocks.
- Reference actual file paths and conventions you observed.
- If you start writing generic FastAPI advice, stop and read the relevant repo file.
- If something is not visible from the files, say so.

Output format:

Return markdown suitable for docs/decisions/comments-feature-plan.md, followed by "Files read" and "Assumptions to verify."

Why the improved version is better: It makes repository reading a requirement and keeps the task as a design review. The plan becomes something a student can critique like a teammate proposal.

Optional follow-up prompt:

Before I save this plan, list which sections are strongest and which are most likely to need human correction.

Common mistake this prompt avoids: Letting the agent start implementing comments or accepting a plan that does not mention actual project files.

5.4C - Critique and compare feature plans

Prompt category: Feature planning / technical critique

When to use it: Use after you have both the generic plan and the repo-grounded Codex plan.

Weak version:

Is this plan good?

Improved version:

I am evaluating two comments-feature plans for Module 5: one generic plan and one repo-grounded Codex App plan.

Grading labels:

- Right: accurate about the existing repo and safe to keep.
- Missing: a specific required detail is absent.
- Needs-Resequencing: the idea may be useful, but the order is wrong for implementation or review.

Task:

1. Grade each section of the repo-grounded plan: Data Model, API Routes, Tests, Frontend Changes, Migration Notes, Open Questions.
2. For each section, give evidence from the plan or actual repo context I provide.
3. Compare the generic and repo-grounded plans in three lines:
 - Biggest difference
 - Plan I would hand to a teammate and why
 - A task shape where generic chat is enough

Constraints:

- Do not implement the feature.
- Do not rewrite the whole plan.
- Do not invent repo facts that are not in the pasted plan or context.

Output format:

1. Section critique table: Section | Label | Evidence | Minimal correction
2. Three-line generic vs repo-grounded comparison

Generic plan:

[PASTE GENERIC PLAN HERE]

Repo-grounded plan:

[PASTE CODEX PLAN HERE]

Repo notes, if needed:

[PASTE ANY FILE EVIDENCE YOU CHECKED YOURSELF]

Why the improved version is better: It creates the exact student work the lecture asks for: critique, not implementation. It also makes the difference between generic and repo-grounded planning obvious.

Optional follow-up prompt:

Ask me three review questions I should answer before any future implementation of this comments feature.

Common mistake this prompt avoids: Saving an agent-generated design document without labeling what is right, missing, or in the wrong order.

Context engineering prompts - Part 5.5

These prompts run the same architecture-document task three ways. The architecture doc matters, but the comparison matters more. Use fresh threads so the outputs are genuinely comparable.

5.5A - Architecture doc with minimal context

Prompt category: Context engineering / Strategy A

When to use it: Use in a fresh Codex App thread as the minimal-context baseline.

Weak version:

Write an architecture doc for this repo.

Improved version:

This is Strategy A: minimal context.

Task:

Produce a one-page architecture doc for this Task Tracker repo using only the minimal task description below. You may inspect repo files as needed, but do not ask me for extra summaries first.

Minimal task description:

Create a concise architecture document for the Task Tracker app.

Required sections:

1. What the app does - one short paragraph.
2. Data model - entities and important fields.
3. Request flow - what happens when a user creates a task.
4. Key files - 5-10 important files, one line each.
5. Conventions - validation, storage, error handling, frontend/backend interaction.
6. Not visible or assumptions - anything you could not confirm.

Constraints:

- One page maximum.
- Output markdown.
- Do not produce code.
- Do not edit app/.
- If you save a file, save only to docs/architecture-A.md after showing me the draft.

Output format:

Return:

1. Draft architecture-A.md
2. Files inspected
3. Assumptions or possible inventions

Why the improved version is better: It intentionally gives Codex little context so students can observe what it discovers correctly and what it invents or overgeneralizes.

Optional follow-up prompt:

Mark any sentence in this draft that could be true of almost any FastAPI app.

Common mistake this prompt avoids: Assuming the first architecture doc is good simply because it is readable.

5.5B - Architecture doc with structured context

Prompt category: Context engineering / Strategy B

When to use it: Use in a new Codex App thread after Strategy A. Provide AGENTS.md and a short file summary to make context structured.

Weak version:

Here is a bunch of project context. Write architecture docs.

Improved version:

This is Strategy B: structured context.

Context:

Here is AGENTS.md:

[PASTE AGENTS.md HERE]

Here is a one-line summary of each important app file:

[PASTE FILE SUMMARY LIST HERE]

Task:

Using the structured context above, produce a one-page architecture doc for the Task Tracker app.

Required sections:

1. What the app does - one short paragraph.
2. Data model - entities and important fields.
3. Request flow - what happens when a user creates a task.
4. Key files - 5-10 important files, one line each.
5. Conventions - validation, storage, error handling, frontend/backend interaction.
6. Not visible or assumptions - anything you could not confirm.

Constraints:

- One page maximum.
- Output markdown.
- Do not produce code.
- Do not edit app/.
- If you save a file, save only to docs/architecture-B.md after showing me the draft.

- Do not add details that are not supported by AGENTS.md, the file summaries, or files you explicitly inspect.

Output format:

Return:

1. Draft architecture-B.md
2. Which context item helped most
3. Any remaining assumptions or unsupported details

Why the improved version is better: It teaches that structured context can improve completeness, but it still requires verification and length control.

Optional follow-up prompt:

Compare this draft against Strategy A. What became more specific, and what became too long or too confident?

Common mistake this prompt avoids: Believing that more context always means better output.

5.5C - Architecture doc with targeted context

Prompt category: Context engineering / Strategy C

When to use it: Use in a new Codex App thread for a narrow and honesty-focused architecture draft.

Weak version:

Only read the important files and write the doc.

Improved version:

This is Strategy C: targeted context.

Read exactly these files and nothing else unless I approve a follow-up:

- app/main.py or the main API entry file
- app/models.py or the main model file
- app/storage.py or the main persistence file

Task:

From only those files, produce a one-page architecture doc for the Task Tracker app.

Required sections:

1. What the app does - one short paragraph.
2. Data model - entities and important fields.
3. Request flow - what happens when a user creates a task.
4. Key files - 5-10 important files, one line each.
5. Conventions - validation, storage, error handling, frontend/backend interaction.
6. Not visible or assumptions - anything you could not confirm.

Special rule:

If something is not visible from the files you read, write "not visible from the files I read" instead of inferring from framework convention.

Constraints:

- One page maximum.
- Output markdown.
- Do not produce code.
- Do not edit app/.

- If a listed file does not exist, tell me the closest actual equivalent and ask before substituting it.
- If you save a file, save only to docs/architecture-C.md after showing me the draft.

Output format:

Return:

1. Draft architecture-C.md
2. Files read
3. Items marked not visible
4. What this targeted strategy likely missed

Why the improved version is better: It teaches that targeted context can produce narrower, more honest output. The repeated "not visible" statements are a feature, not a failure.

Optional follow-up prompt:

What questions would you need to answer before turning this into a full onboarding architecture document?

Common mistake this prompt avoids: Rewarding broad confident output more than honest limitations.

5.5D - Compare context strategies

Prompt category: Context engineering / comparison and rule-making

When to use it: Use after completing architecture-A.md, architecture-B.md, and architecture-C.md.

Weak version:

Which architecture doc is best?

Improved version:

I ran the same architecture-document task with three context strategies.

Strategy A: minimal context

Strategy B: structured context using AGENTS.md and file summaries

Strategy C: targeted context using a small set of anchor files

Task:

Compare the three outputs and help me write the comparison log for docs/architecture.md.

For each strategy, identify:

- What it got right.
- What it got wrong, missed, or invented.
- Which task shape it is best suited for.

Then help me write:

1. A verdict: which strategy I chose for the final architecture doc and why.
2. A two-sentence context-engineering rule in the form: "For task shape X, I use strategy Y because Z."

Constraints:

- Use only the three drafts I paste below.
- Do not invent repo facts.
- Do not rewrite the entire architecture document unless I ask.
- Make the comparison specific; avoid generic statements such as "context matters."

Output format:

1. Strategy comparison table
2. Verdict paragraph
3. Two-sentence context-engineering rule

Architecture A:

[PASTE ARCHITECTURE-A HERE]

Architecture B:

[PASTE ARCHITECTURE-B HERE]

Architecture C:

[PASTE ARCHITECTURE-C HERE]

Why the improved version is better: The lecture says the comparison is the real lesson. This prompt turns the experiment into a reusable rule for future tasks.

Optional follow-up prompt:

Make the final two-sentence rule more specific by naming the exact task shapes from my course work.

Common mistake this prompt avoids: Leaving with three generated docs but no transferable context-engineering principle.

Personal AI playbook prompts - Part 5.6

These prompts support the final course artifact. Codex can scaffold and review, but the playbook must be written in the student's own voice and backed by course evidence.

5.6A - Draft a playbook template only

Prompt category: Personal playbook / template scaffold

When to use it: Use at the start of Part 5.6. The AI should create structure, not fill in your rules.

Weak version:

Write my AI playbook.

Improved version:

Draft a one-page personal AI coding playbook template for me to fill in.

Sections:

1. When I reach for AI first
2. When I do not reach for AI
3. My non-negotiables
4. My review rules
5. What I am still figuring out

For each section, provide 2-3 empty bullet placeholders that show what kind of thing belongs there. Do not fill in my actual rules.

End with a Decision Card in exactly this format:

- For a new feature I reach for: ____
- For a code review I reach for: ____
- For debugging I reach for: ____
- For infrastructure I reach for: ____
- I will never paste ____ into an AI tool.
- My one rule is: ____

Constraints:

- Template only.
- Do not invent examples from my course experience.
- Do not write generic advice inside the blanks.
- Keep it short enough for a one-page docs/ai-playbook.md.

Why the improved version is better: It prevents the most common playbook failure: letting AI fill the rules. The student owns the final content.

Optional follow-up prompt:

Review your template and remove any rule-like content you accidentally filled in for me.

Common mistake this prompt avoids: Submitting a polished AI-written playbook that could have been written by any student.

5.6B - Review playbook for evidence and voice

Prompt category: Personal playbook / student-owned review

When to use it: Use after writing your own draft of docs/ai-playbook.md.

Weak version:

Make this playbook better.

Improved version:

I wrote the playbook below myself. Review it for evidence and student voice, but do not rewrite it for me.

Task:

Check whether my playbook meets the Module 5 quality bar:

- It is one page or close to one page.
- It uses my own voice, not generic AI-policy language.
- Each rule is backed by a specific course incident or observation.
- It mentions practical review habits, not vague aspirations.
- The Decision Card is fully filled in.
- The never-paste rule names a concrete type of data.
- The 30-day re-read commitment is present.

Constraints:

- Do not produce a replacement playbook.
- Do not add evidence I did not provide.
- Suggest minimal edits only.
- If a rule lacks evidence, mark it "needs course evidence".

Output format:

Return a checklist table with columns:

Requirement | Present / Missing | Evidence from my draft | Minimal edit

My playbook draft:

[PASTE YOUR PLAYBOOK HERE]

Why the improved version is better: It uses AI as a reviewer while preserving the course principle that the final artifact must be personal and evidence-backed.

Optional follow-up prompt:

Ask me five questions that would help me make this playbook more specific without writing it for me.

Common mistake this prompt avoids: Polishing the playbook prose while leaving the rules generic, unsupported, or not actually personal.

5.6C - Decision Card challenge

Prompt category: Tool choice / final reflection

When to use it: Use when you are stuck filling the Decision Card. The AI should ask forcing questions, not choose for you.

Weak version:

Fill in my Decision Card.

Improved version:

Do not fill in my Decision Card for me.

Instead, ask me one forcing question for each blank:

- For a new feature I reach for: ____
- For a code review I reach for: ____
- For debugging I reach for: ____
- For infrastructure I reach for: ____
- I will never paste ____ into an AI tool.
- My one rule is: ____

Use the tool shapes from the course to make the questions concrete:

- Cursor for larger implementation loops.
- GitHub Copilot for continuous editor pairing.
- Claude Code for terminal delivery work.
- Codex App for desktop review, planning, and governance.
- General chat for reasoning, drafting, and reflection without repo access.

Constraints:

- Ask questions only.
- Do not recommend one tool as universally best.
- Do not fill any blank for me.
- Push me to answer from evidence, not preference or hype.

Output format:

Return six numbered questions, one for each Decision Card blank.

Why the improved version is better: The Decision Card is the one part AI cannot own. This prompt helps students think without letting the model decide.

Optional follow-up prompt:

After I answer the six questions, check whether any answer is too generic or not tied to a course experience.

Common mistake this prompt avoids: Choosing tools by habit or hype instead of matching the interface to the task shape.